Diversity-Oriented Testing for Competitive Game Agent via Constraint-Guided Adversarial Agent Training

Xuyan Ma, Yawen Wang, Junjie Wang, *Member, IEEE*, Xiaofei Xie, *Member, IEEE*, Boyu Wu, Yiguang Yan, Shoubin Li, Fanjiang Xu, and Qing Wang, *Member, IEEE*

***Abstract*—Deep reinforcement learning has achieved remarkable success in competitive games, surpassing human performance in applications ranging from business competitions to video games. In competitive environments, agents face the challenge of adapting to continuously shifting adversary strategies, necessitating the ability to handle diverse scenarios. Existing studies primarily focus on evaluating agent robustness either through perturbing observations, which has practical limitations, or through training adversarial agents to expose weaknesses, which lacks strategy diversity exploration. There are also studies which rely on curiosity-based mechanism to explore the diversity, yet they may lack direct guidance to enhance identified decision-making flaws. In this paper, we propose a novel diversity-oriented testing framework (called AdvTest) to test the competitive game agent via constraint-guided adversarial agent training. Specifically, AdvTest adds constraints as the explicit guidance during adversarial agent training to make it capable of defeating the target agent using diverse strategies. To realize the method, three challenges need to be addressed, i.e., what are the suitable constraints, when to introduce constraints, and which constraint should be added. We experimentally evaluate AdvTest on the commonly-used competitive game environment, StarCraft II. The results on four maps show that AdvTest exposes more diverse failure scenarios compared with the commonly-used and state-of-the-art baselines.**

***Index Terms*—Adversarial agent, constrained reinforcement learning, testing diversity.**

Received 8 February 2024; revised 8 October 2024; accepted 31 October 2024. Date of publication 5 November 2024; date of current version 10 January 2025. This work was partially supported by the Youth Innovation Promotion Association Chinese Academy of Sciences, National Natural Science Foundation of China under Grant 62232016 and Grant 62072442, Basic Research Program of ISCAS under Grant ISCAS-JCZD-202304, Major Program of ISCAS under Grant ISCAS-ZD-202302, and Innovation Team 2024 ISCAS (No. 2024-66), the National Research Foundation, Singapore, and the Cyber Security Agency under its National Cybersecurity R&D Programme under Grant NCRP25-P04-TAICeN. Recommended for acceptance by T. Yue. *(Xuyan Ma and Yawen Wang contributed equally to this work.) (Corresponding authors: Junjie Wang; Qing Wang.)*

Xuyan Ma, Yawen Wang, Junjie Wang, Boyu Wu, Yiguang Yan, Shoubin Li, Fanjiang Xu, and Qing Wang are with the State Key Laboratory of Intelligent Game, Institute of Software Chinese Academy of Sciences, and University of Chinese Academy of Sciences, Beijing 100190, China (e-mail: maxuyan2021@iscas.ac.cn; yawen2018@iscas.ac.cn; junjie@iscas.ac.cn; yanyiguang2021@iscas.ac.cn; shoubin@iscas.ac.cn; fanjiang@iscas.ac.cn; wq@iscas.ac.cn; boyu_wu2021@163.com).

Xiaofei Xie is with Singapore Management University, Singapore 188065 (e-mail: xiaofei.xfxie@gmail.com).

Digital Object Identifier 10.1109/TSE.2024.3491193

I. Introduction

COMPETITIVE games present scenarios where multiple agents interact, each striving to optimize their individual objectives, defeat opponents, and emerge victorious. Deep Reinforcement Learning (DRL) equips agents with the ability to learn from extensive data, adapt to dynamic environments, and optimize complex objectives, thus has emerged as a pioneering methodology for addressing and comprehending competitive games [1]. Its applications span a wide spectrum, from achieving super performance in board games to addressing critical contexts like military conflicts, business competitions, political science, and resource allocation tasks [2].

However, the training process in competitive settings is intricate. The policy or agent should not only adapt to a static set of rules but also contend with the constantly shifting strategies of adversaries. If not handled effectively, this can lead to agents that are fragile and struggle to generalize across diverse scenarios [3]. In safety- and security-critical contexts such as military conflicts or business competitions, a non-robust agent can have dire consequences, ranging from financial losses to risks to human life. In these environments, it is not only necessary to understand the ability of the agent to complete the task, but also to discover the failure scenarios of the agent and analyze the reasons for its failure. Consequently, there is an urgent need for testing the agent's robustness in competitive games.

Many studies have explored this field. One approach involves perturbing the agent's observations (i.e., the game environments) at specific time steps [4], [5], thereby inducing suboptimal actions from the agent's policy, leading to its failure. However, these methods may not be practical in real-world settings, as altering physical environments, such as introducing pixel noise to input images, is often challenging. Moreover, this approach may not effectively uncover inherent decision-making flaws that are unrelated to such digital perturbations. Another line of research focuses on training adversarial agents to defeat the opponents in competitive games [6], [7], [8], [9], offering a potential solution to the aforementioned challenges. The adversarial agent trained in this manner can expose the decision-making weaknesses of the target agent. However, a significant challenge lies in the limited diversity of identified weaknesses. Typically, adversarial agents are trained to defeat the opponents. Their strategies are limited, focusing only on

finding and exploiting the weaknesses that are easiest to find, while potentially overlooking others.

Some research underscores the importance of testing diversity, as seen in BehAVExplor [10], Wuji [11], EMOGI [12] and RPG [13]. However, these methods primarily rely on curiosity-based approaches to enhance diversity, which may not provide direct guidance. Specifically, in complex tasks and environments like StarCraft, while curiosity-based methods can identify decision-making flaws through curiosity scores, diversity is limited because such indicators cannot explicitly guide agent's behaviors towards a meaningful and targeted direction. Taking a competitive setting in our experiment, which involves two marines against one zealot, as an example, existing techniques such as EMOGI are mostly exploring different cases under the same winning strategy, e.g., the two marines collaborate and defeat the zealot to ensure a secure victory, yet cannot find cases under other strategies, such like one marine adopting a defensive stance while the other marine applies conservative strategy on the opponent to pursue a narrow victory margin. This raises the question of whether we can employ more direct guidance (e.g., conservative strategy to keep small health difference) specific to the task to enhance decision-making diversity.

In this paper, we aim to explore a testing approach that can explicitly guide the training of an adversarial agent capable of defeating the target agent using diverse strategies. Reward randomization [13] and fuzz testing [10], [14] can potentially induce diverse strategies for agents, but their diversity guidance is random and not explicit. We add *constraints* during training, serving as explicit guidance to train adversarial agent with different strategies. The rationale is that the constraint gives the agent a strategic direction, and when the agent's behavior violates the constraint, it will be punished, so the agent will tend to change its strategy to conform to the constraint. Such adversarial agent could reveal diverse weaknesses of the target agent. Note that, in the following paper, we call the agent under test as **target agent**, and our trained adversarial agent as **testing agent** which is for testing the target agent.

To achieve this, we need to address three primary challenges: 1)***WHAT*** *are the suitable constraints*: There are many factors that can influence the process of competitive game. We expect to identify a suitable set of constraints that can be easily implemented and have great impact on agent's behavioral diversity. Therefore, the constraint set needs to be well-designed. 2) ***WHEN*** *should we introduce constraints during adversarial agent training*: The agent tends to gradually find the most effective policy as training goes on, resulting in a decrease in diversity. If we can add constraints at the point of low diversity, it would be more effective and low-overhead [15]. Therefore, it is important to investigate the timing of adding constraints. 3) ***WHICH*** *constraint should be added at specific time*: Adding different constraints at the same time has different effects. If the added constraint is consistent with the agent's current behavior, it is difficult to change its strategy. Therefore, in order to increase the behavioral diversity, it is important to decide which constraint to add at a specific time.

To tackle these challenges, this paper introduces a novel diversity-oriented testing framework, called AdvTest, which is built upon the concept of training an adversarial agent with constraints as explicit guidance. These added constraints are intended to compel the adversarial agent to explore diverse winning strategies, thus revealing a range of weaknesses (i.e., distinct failure scenarios) of the target agent. To address the *WHAT* issue, we begin by extracting insights from competitive game manuals and online discussions among users. From these sources, we manually identify five pivotal factors that can influence the progression and patterns of competition. Subsequently, we distill these insights into seven specific constraints to provide explicit guidance during training. To tackle the *WHEN* issue, we introduce a *Diversity Check* module. This module employs the normalized Hamming distance to quantify the similarity between state sequences generated by competitive environment of varying lengths. Regarding the *WHICH* issue, we develop a *Constraint Selection* module. This module maintains a record of the diversity gain achieved by each constraint selection throughout the testing agent training competition history, offering a global perspective on constraint preference. Additionally, it incorporates constraint indicators derived from real-time competitive behaviors, providing a local viewpoint on constraint preference. This dual method helps determine the most effective constraints to introduce during the training process.

To evaluate the effectiveness of AdvTest, we implement AdvTest on the popular and open-source competitive environment, StarCraft II [16], and conduct experiments on four maps with different troop configurations. The results show that AdvTest significantly outperforms the baselines in discovering diverse failure scenarios. Specifically, compared with the state-of-the-art technique EMOGI, AdvTest increases the number of unique failure scenarios by 5.6% to 66.7%, increases the average distance of generated failure scenarios by 15.4% to 88.3% and enhances the state coverage by 32.1% to 54.2% on the four maps respectively. Besides, we conduct an ablation experiment, which verifies both global and local preference in *Constraint Selection* module contribute to the performance improvement. Furthermore, we summarize eight types of high-level battle patterns, and the results show that failure scenarios discovered by AdvTest can cover more types of patterns than baselines.

The contributions of this paper are as follows:

- The first attempt to the best of our knowledge utilizing the concept of training adversarial agents with constraints as explicit guidance for testing agent of competitive game.
- A diversity-oriented testing framework AdvTest with constraint-guided adversarial agent training, which addresses the challenges: what are the suitable constraints, when to introduce the constraints, and which constraint to be added.
- Experimental evaluation of effectiveness of AdvTest on four maps of StarCraft II with promising performance, outperforming four baselines.
- Public released source code of AdvTest to facilitate the replication and extension of this study[1].

[1]Details are in our website: https://anonymous.4open.science/r/TSE-AdvTest

II. Background and Motivational Study

A. Background

1) Adversarial Agent Training: Adversarial agent (i.e., testing agent) training is to train an adversarial policy (agent) to compete against the opponent (i.e., target agent) in a Markov game. The adversarial agent and target agent are denoted by subscript α and ν, respectively. The game $M = (S, (A_\alpha, A_\nu), T, (R_\alpha, R_\nu))$ consists of state set S, action sets A_α and A_ν, and a joint state transition function $T : S \times A_\alpha \times A_\nu \to \Delta(S)$, where $\Delta(S)$ is a probability distribution on S. For each agent (α and ν), the reward function $R : S^{(t)} \times A_\alpha \times A_\nu \times S^{(t+1)} \to \mathbb{R}$ depends on the current state $S^{(t)}$, next state $S^{(t+1)}$ and both players' actions. Each player wishes to maximize their sum of rewards.

When testing the target agent, it is fixed and black-box, corresponding to the common case of a pre-trained model. Then the Markov game M reduces to a single-player Markov Decision Process (MDP) question: $M_\alpha = (S, A_\alpha, T_\alpha, R'_\alpha)$ that the adversarial agent needs to solve. The state and action space of the adversarial agent are the same as in M, while the transition and reward function have the target agent π_ν embedded: $T_\alpha(s, a_\alpha) = T(s, a_\alpha, a_\nu)$ and $R'_\alpha(s, a_\alpha, s') = R'_\alpha(s, a_\alpha, a_\nu, s')$. The action of target agent is sampled from policy π_ν: $a_\nu \sim \pi_\nu(\cdot \mid s)$. The goal of the adversarial agent is to find a policy π_α maximizing the sum of rewards: $\sum_{t=0}^{\infty} \gamma^t R_\alpha(s^{(t)}, a_\alpha^{(t)}, s^{(t+1)})$.

2) Constrained Reinforcement Learning: Constrained reinforcement learning [17] is often used to ensure the safety of the agent [18], such as the safety requirements (e.g., traffic regulations) of the ego-vehicle in autonomous driving system. It constrains the agent's behavior by setting penalty or restricting action space, which lets the agent strike a balance between maximizing the reward and violating the constraints. Specifically, the constraint set is defined by a set of cost functions $C_i \in \{C_1, ..., C_m\}$ like the usual reward, and the limits $\{d_1, ..., d_m\}$. The expected discounted return of policy π with respect to cost function C_i is defined as: $C_i(\pi) = E_{\tau\sim\pi}[\sum_{t=0}^{\infty} \gamma^t C_i(s^{(t)}, a^{(t)}, s^{(t+1)})]$. As a result, the goal is to maximize reward $R(\pi)$ while meeting the cost $C_i(\pi) \le d_i$, $\forall i \in [m]$:

$$\pi^* = arg \max_{\pi\in\Pi_\theta} R(\pi) \quad s.t.\ C_i(\pi) \le d_i, \forall i \in [m] \qquad (1)$$

where $s^{(t)}$ and $a^{(t)}$ refer to the state and the action sampled from π at time t, respectively, and $s^{(t+1)}$ refers to the state at the next time obtained from $T(s^{(t)}, a^{(t)})$.

B. Scenario Description

A scenario is a temporal sequence of scenes, where each scene is a snapshot of the environment including the static and dynamic objects. Therefore, a scenario can be described by a series of configurable static and dynamic attributes. Static attributes set the static objects of the scenario, such as the map, obstacle, etc. Dynamic attributes define the state (e.g., position, orientation, etc.), trajectory, and behavior (e.g., move, attack, stop, etc.) of dynamic objects. Scenario observation is a sequence of state of all objects at each time step in the simulation environment, including the target agent, testing agent and other environment elements. The failure scenario refers to the scenarios where the target agent makes the undesirable decisions and ultimately fails.

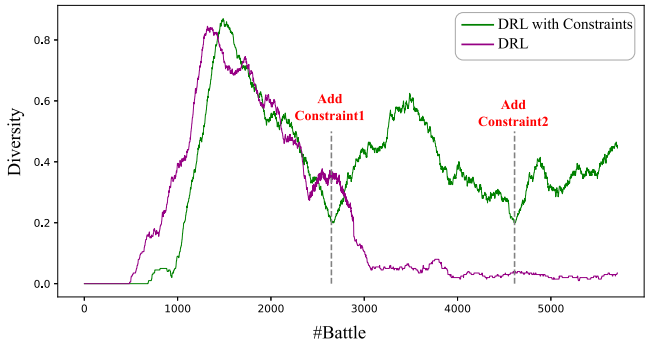


Fig. 1. Motivational study. Constraint1 represents reducing the attack range of testing agent. Constraint2 represents constraining the health difference between the two-sided agents being small range.

C. Motivational Study

Inspired by constrained reinforcement learning, we seek to explore the ability of constraints to change the behavior of agents and to further discover diverse failure scenarios. We firstly explore the capability of DRL in revealing failure scenarios and strategy diversity of target agent. Fig. 1 demonstrates the diversity trend with the number of battles increases, where the diversity is depicted by the number of unique failure scenarios of target agent. We find that the diversity (purple line) goes up and then down, which illustrates that the testing agent tend to explore a set of winning strategies at the beginning so the diversity rises at first, but then it would figure out the most winnable strategy as it gets to understand the opponent, which leads to the decrease of diversity.

We expect an explicit way to guide the testing agent to explore a more diverse set of winning strategies. To this end, we try to add a constraint (i.e., reducing the attack range of testing agent) on the original DRL policy at about $2,600^{th}$ battle, we find an obvious rise in diversity (green line) after that. After a period of rising, it begins to decrease again at about $3,500^{th}$ battle. Then we replace the original constraint with another (i.e., constrain the health difference between the two-sided agents being small range) at about $4,600^{th}$ battle, then we find some new failure scenarios again.

Fig. 2 presents an example of battle patterns before and after adding constraints, as well as effects brought by different constraints being added, where testing agent controls soldier M_L and M_R, while target agent controls soldier Z. The grid above the soldier's head indicates the health value. The arrow indicates the direction of movement. Before adding any constraint as shown in Fig. 2(a), M_L maintains a considerable distance from Z, and M_R also runs away from Z at the same time to minimize the risk of being attacked, thereby increasing their chances of winning the battle. After adding constraint1 (constrain the attack range being small) as shown in Fig. 2(b), M_L and M_R

(a) Original (b) Under constraint1 (c) Under constraint2

Fig. 2. Example of changing battle patterns by adding constraints.

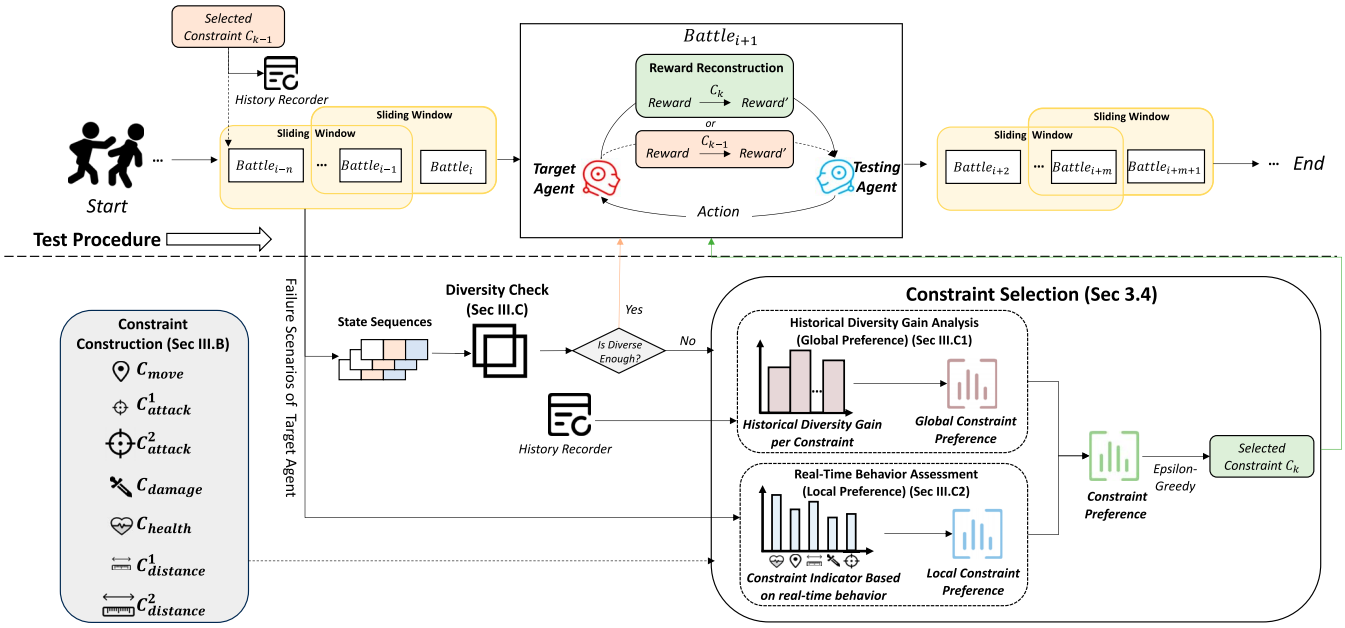


Fig. 3. Overview of AdvTest.

move closer to attack Z, which allows for the exploration of Z's capabilities in near-range defense. Similarly, before adding constraint2, there is a significant health difference between M_L/M_R and Z due to their focus on swiftly winning the battle and opting for aggressive attacks. However, with the inclusion of constraint2 (constrain the health difference between the two-sided agents being small range) as depicted in Fig. 2(c), M_L and M_R adopt a more conservative strategy in attacking Z. It is shown that the health difference between M_R and Z is small. This shift facilitates the exploration of Z's ability to endure persistent attacks.

Through the above observations, we find adding constraints to guide the training process can force the testing agent to explore a diverse set of winning strategies, thus inducing more target agent's diverse strategies. This motivates us to design a technique with explicit constraint-guided adversarial agent training to find diverse failure scenarios of target agent. In this motivational study, what are the suitable constraints, when to introduce the constraints, and which constraint to be added are all decided empirically. We also try some constraints that are consistent with the agent's recent behavior, and find that those constraints cannot change the agent's behavior significantly. For example, when the distance between agents is originally small, we cannot increase diversity by constraining them to reduce the distance further. To sum up, according to the motivational study, these three aspects (i.e., *WHAT*, *WHEN* and *WHICH*) should be well-designed for better testing, and our proposed approach will address these three challenges.

III. Approach

A. Overview

This paper proposes AdvTest, a diversity-oriented testing framework for competitive game agent with the constraint-guided adversarial agent training. As illustrated in Fig. 3, AdvTest trains a testing agent with the goal of finding more and more diverse failure scenarios of the target agent, i.e., defeat the target agent and meanwhile explore its strategy diversity. To achieve this, AdvTest needs to address the following three challenges.

To tackle the first challenge ***what are the suitable constraints***, we manually identified five key factors that affect the process and patterns of competition from the competitive game manual and user discussions. These factors are distilled into seven specific constraints that are used to guide training of the testing agent. Regarding the second challenge ***when should***

we introduce constraints during adversarial agent training, we design *Diversity Check* module that uses normalized Hamming distance to quantify the similarity between failure scenarios of varying lengths and further determines whether the diversity is enough at the current moment. To address the third challenge ***which constraint should be added at a specific time***, we develop *Constraint Selection* module. It records the diversity gain obtained by each constraint as the constraint's global preference. Moreover, it collects the values of constraint indicators based on the agent's real-time behavior, providing local preference for constraints. This dual approach can adjust the strategy of the testing agent during training.

Additionally, AdvTest designs a sliding window mechanism to make the whole process more effective, i.e., focusing on the most recent battles. In detail, it sets up a sliding window containing a certain number of battles (i.e., window size). With the finish of a new battle, the *Diversity Check* would determine whether the recent battles are diverse enough or not. If yes, the window would slide forward. Otherwise, the *Constraint Selection* begins to select a constraint. The k-th selected constraint C_k would remain in effect until the next constraint is selected. For the battles in the C_k working region, AdvTest would constrain the testing agent's policy with C_k by reconstructing the reward function.

B. Constraint Construction

We begin with the competitive game manuals and online discussions among users, and manually identify five pivotal factors that can influence the progression and patterns of the competition.

- Factors related with the agent itself's capability
  - Movement Range: narrowing the movement range limits the agent's moving capability.
  - Attack Range: reducing or increasing the attack range can directly change the attack capability of the agent.
  - Damage Value: reducing the damage value limits the agent's fight capability.
- Factors related with the interactive status of two agents
  - Health Difference: it influences the fight strategy, e.g., to conservatively defeat the opponent or beat at one stroke aggressively (maintain small/big health difference).
  - Distance: the distance between agents affects the attack strategy of agents, e.g., "far attack" or "close attack".

We then distill these factors into seven specific constraints as explicit guidance during training, to facilitate the discovery of diverse scenarios. The constraints employed in this work are based on the general design concepts of competitive games, and reflect the common situations in the game regression. In this work, three authors together construct the constraints and take about half day. And these constraints can be reused directly, or with minimal modifications, for testing other competitive environments.

More specifically, each factor can derive two constraints, i.e., from the perspectives of lower bound and upper bound. Yet some of them have the same effects on the battle result (e.g., increasing damage value has the same goal with training the agent to win) or are not allowed by the game settings (e.g., enlarging movement range causes the agent to leave the map boundary), hence we would not include them in the constraints. In this way, we totally come up with seven constraints derived from the above five factors.

- **Movement within smaller range** C_{move}: constrain the bound of the movement range to 50% of its default value. Because the target agent has no limited range of movement, this movement restriction can potentially cause the testing agent not being able to get close to the target agent. The testing agent could not launch the attack which is different from its usual behavior aggressively pursuing victory in the game.
- **Attack within smaller range** C^1_{attack}: constrain the attack range to 50% of its default value. This constraint would potentially cause the target agent to move closer to the opponent in order to be able to launch attack.
- **Attack within larger range** C^2_{attack}: constrain the attack range to 150% of its default value. This constraint would potentially make the agents to choose "far attack" since it can protect itself from being attacked.
- **Smaller damage value** C_{damage}: constrain the damage value of each attack to less than 50% of its default damage. This forces the testing agent to adopt a more conservative strategy in defeating the target agent, potentially exploring the target agent's different strategies.
- **Smaller health difference between two-sided agents** C_{health}: constrain the health difference between the two-side agents being less than 50% of the testing agent's maximum health value. This would encourage the testing agent to choose a conservative strategy in order to maintain a small health difference between itself and the target agent.
- **Smaller distance between two-sided agents** $C^1_{distance}$: constrain the distance between two-sided agents being less than 50% of the attack range. This encourages the testing agent to move closer to the target agent, adopting a "close attack" strategy.
- **Larger distance between two-sided agents** $C^2_{distance}$: constrain the distance between two-sided agents being more than 50% of the attack range. This encourages the testing agent to keep a distance from the target agent, employing a "far attack" strategy.

Note that some constraints might have similar effects, e.g., attack within smaller range and smaller distance between two-side agents would both encourage the agents to battle in a close manner. Yet different constraints would have their specific emphasis, and the experimental results also reveal the necessity of each constraint.

C. Diversity Check

The goal of *Diversity Check* is to address the *WHEN* issue. Because our aim is to find failure scenarios of the target agent, we only consider the scenarios where it loses the battle and measure whether the recent failure scenarios are different from the existing scenarios ever since the testing begins. To select the

most recent battles, we define a sliding window. If the recent failure scenarios are not diverse enough, which implies the testing agent defeats the target agent in similar ways, AdvTest would turn to *Constraint Selection* to add another constraint to boost the diversity. Otherwise, AdvTest would slide the window forward and begin a new battle.

The challenge here is that the scenarios are state sequences with widely varying sizes, which makes commonly-used similarity measure might fail to take effect. For example, when the testing agent attacks the opponent with an aggressive strategy, the battle would end quickly. Then the state sequences it produces are much shorter than that produced by a testing agent with a conservative strategy. Therefore, we employ the normalized Hamming distance to measure the similarity between the state sequences.

We begin with the details of the state sequence, then introduce how the diversity is measured. The state sequences within the sliding window can be denoted as $Q = \{s_1, s_2, ..., s_w\}$, where w is the total number of battles within the window (i.e., window size). And each s_i is the state sequence of the i-th battle, which is a $m \times n$ matrix, indicating each battle has m frames and the observation in each frame is a n-dimensional vector. As Sec. II-B mentioned, the observation contains the agents' information such as health, position, and other environmental parameters.

Since some state sequences within the sliding window can be quite similar with the existing ones, we first filter out these similar sequences and only utilize the remaining ones for finally determining the diversity score. In detail, for a state sequence s generated by $Battle_i$ within the sliding window, we calculate the minimum distance with all other existing ones $d_s = \min_{s' \in SEQ} dis(s, s')$, where SEQ represents the set of state sequences generated from $Battle_1$ to $Battle_{i-1}$.

The state sequences have different lengths due to different execution time. Therefore, for the distance calculation dis, the existing time sequence distance metrics, such as Dynamic Time Warping [19], may not work well in our situation. To this end, the normalized Hamming distance between two state sequences is calculated as follows:

$$dis(s, s') = \frac{Hamming(s, s') + |len(s) - len(s')|}{max(len(s), len(s'))} \quad (2)$$

where $Hamming(s, s')$ computes the Hamming distance [20] between the segments of common length in s and s', i.e., the forehand states in each sequence. The denominator of Equation 2 is the maximum length of the two state sequences, which takes into account the length of the state sequence and normalizes the distance to the interval $[0, 1]$.

The final diversity is calculated by the frequency of newly emerging failure sequence within the sliding window. We utilize a pre-defined similarity threshold θ_s to filter out the similar sequence, i.e., when $d_s > \theta_s$, it indicates the state sequence is different from existing ones, and will be treated as a newly emerging failure sequence. We then calculate the frequency of newly emerging failure sequences within the scope of sliding window as: $freq = \#seq/w$, where $\#seq$ represents the number of newly emerging failure sequences and w is the window size. When $freq$ is less than the pre-defined frequency threshold θ_f, the *Diversity Check* considers it is time to add constraints.

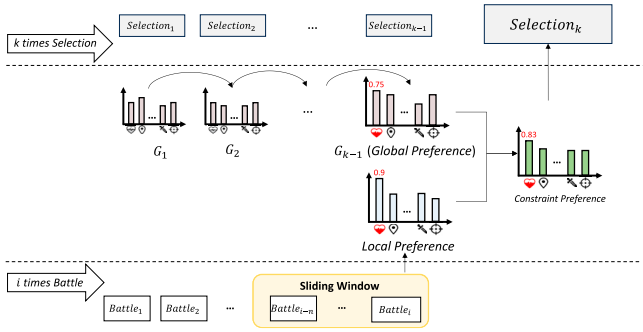


Fig. 4. Schematic diagram of Constraint Selection.

D. Constraint Selection

After *Diversity Check* module considers it is time for introducing constraints, this *Constraint Selection* module would determine which constraint should be added. First, the performance of historical selected constraints can act as good indicators for future decision. Hence, we record the diversity gain per constraint in the selection history to represent the global constraint preference. However, relying solely on such global perspective is not enough, because there are no records in the initialization phase, and the performance can also be influenced by the recent behaviors of testing agent. Therefore, we also record the real-time constraint violations to represent the local constraint preference. AdvTest makes the final constraint selection jointly considering both global and local preference. Fig. 4 illustrates an example of the process described above.

1) Constraint Selection With Global Preference: AdvTest could collect the global information, which contains the history of each constraint selection and the corresponding diversity gains brought by each selection. The goal of AdvTest is to maximize cumulative diversity through certain strategies of selecting constraints. We analogize this task to the Multi-armed Bandit problem, which originally aims to maximize cumulative rewards through certain strategies of pulling arms [21]. We can treat each constraint as each arm on a bandit machine, and selecting a constraint as selecting the next arm to pull.

Alg. 1 presents the process of constraint selection with global preference. It takes constraint set and the times of constraint selection as input. Each time a constraint c is selected by the ϵ-greedy selection (line 3), which would be introduced in Sec. III-D3, AdvTest would count the number of diverse failure scenarios before the next constraint selection as the diversity gain brought by c (line 4). Then it would update the average reward $R(c)$ of the constraint c based on the current reward (line 5), and prepare for the next selection. The algorithm finally returns the reward vector $R(c_i)$ as the global constraint preference (line 8), where a higher reward means higher preference of selecting c_i. We then normalize $R(c_i)$ to make each element in the range $[0, 1]$, which eliminates the effect brought by different orders of magnitude.

Algorithm 1 Constraint Selection with Global Preference

Input: Constraint Set C, the T^{th} Selection
Output: Rewards $R(c_i)$
1: $\forall c_i \in \{c_1, ..., c_n\}$, $R(c_i) = 0$, $count(c_i) = 0$
2: **for** t = 1, 2,..., T **do**
3: $c = \epsilon$-*GreedySelection()*
4: $v = Reward(c)$
5: $R(c) = \frac{R(c)*count(c)+v}{count(c)+1}$
6: $count(c) = count(c) + 1$
7: **end for**
8: **return** $R(c_i)$

2) Constraint Selection with Local Preference: To represent the local constraint preference, AdvTest would record real-time constraint violations to calculate the recent constraint indicators. The general idea is to let AdvTest select the constraint which is strongly violated recently based on the corresponding indicator. This would potentially lead AdvTest to select constraints that have not been selected recently.

We define five constraint indicators corresponding to five factors mentioned in Sec. III-B, based on which the local constraint preference will be derived. The details of constraint indicators are listed below.

- I_{move}: The number of times the testing agent leaves the restricted movement range over given time steps, corresponding to constraint C_{move}, ranging in $[0, max_m]$, where max_m is the number of time steps.
- I_{attack}: The average actual attack range over given time steps, corresponding to constraint C^1_{attack} and C^2_{attack}, ranging in $[0.5r, 1.5r]$, where r is the default attack range.
- I_{damage}: The average damage value of testing agent over given time steps, corresponding to constraint C_{damage}, ranging in $[0, max_d]$, where max_d is the default maximum damage.
- I_{health}: The average difference between the health value of the two-sided agents over given time steps, corresponding to constraint C_{health}, ranging in $[0, max_h]$, where max_h is the maximum health value.
- $I_{distance}$: The average distance between two-sided agents over given time steps, corresponding to constraint $C^1_{distance}$ and $C^2_{distance}$, ranging in $[0, d]$, where d is the distance between the two farthest points on the map.

Given the statistics of indicators above, we perform min-max normalization ($x_{new} = \frac{x - x_{min}}{x_{max} - x_{min}}$) on these indicators in terms of their upper and lower bounds, and obtain a normalized value in $[0, 1]$. For the indicators corresponding to one constraint, we directly use this normalized value as its local preference. And for the indicators corresponding to two constraints, we treat this normalized value as the local preference for the first constraint (i.e., $C^1_{distance}$ and C^1_{attack}). We then exchange the x_{min} and x_{max} to compute a second normalized value and treat it as the local preference for the second constraint (i.e., $C^2_{distance}$ and C^2_{attack}). Taking $C^1_{distance}$ and $C^2_{distance}$ as an example, if the distance between two agents is getting larger, the preference of $C^2_{distance}$ decreases while the preference of $C^1_{distance}$ increases. In other words, when AdvTest finds the distance is too large, it tends to select $C^1_{distance}$ to bring them close, and vice versa.

3) Epsilon-Greedy Selection: Through above two modules, we obtain a global constraint preference and a local constraint preference, where each element corresponds to one constraint. Then we weight global and local preferences with the same weight (i.e., 0.5) to obtain a probability vector Pr, where each element represents the combined probability of selecting one constraint. The greater value indicates higher probability of being selected.

Given the vector Pr, we perform constraint selection based on a widely-used algorithm ϵ-greedy, to increase exploration, where the constraint is selected randomly with the probability of ϵ or greedily (i.e., the constraint with the largest Pr value is selected) with the probability of 1-ϵ. Following common practice [22], we set ϵ as 0.1. When a new constraint is selected, AdvTest would replace the old constraint with the new one, i.e., adding a discount factor to reconstruct the reward to penalize the agent's behaviors which violates the constraints, following the paradigm of constrained RL as described in Formula 1.

4) Example of Constraint Selection: Fig. 4 shows an example of constraint selection. The global preference is iterated and updated based on historical constraint selection. And using the collected state sequence of the battles in the sliding window, the constraint indicators and local preference can be counted, which reflects the real-time behavior of testing agent. While making k-th selection, the global preference G_k is updated by G_{k-1} (details are in Alg. 1). The local preference is calculated by the state sequences of $Battle_{i-n}$ to $Battle_i$. Therefore, we select current constraint based on the weighted average of global preference G_k and the local preference. In the example in Fig. 4, C_{health} has the best performance (i.e., 0.83), so it is selected at the k-th selection.

IV. Experiment Design

A. Research Questions

We consider the following research questions:

- **RQ1**: Can AdvTest effectively find more and diverse failure scenarios in competitive games?
- **RQ2**: How useful is global preference and local preference for selecting constraints?
- **RQ3**: Is each constraint necessary in AdvTest?

B. Experimental Environment

To verify the effectiveness of AdvTest, we choose one of the most popular real-time strategy games StarCraft II [16] as the experimental environment and use the latest framework PyMARL2 [23] for interacting with the game environment and training DRL algorithms. PyMARL2 includes implementations of basic multi-agent reinforcement learning algorithms.

There are a total of 26 maps ($2m\_vs\_1z, 3s\_vs\_3z, etc.$) and 10 types of soldiers (Marine, Stalker, Zealot, etc.) provided in the environment. Different maps have different terrains,

TABLE I
EXPERIMENTAL MAPS

Map Name	Testing	Target	Test Duration (Min)	Similarity Threshold θ_s
$2m\_vs\_1z$	2 Marines	1 Zealots	30	0.3
$3m$	3 Marines	3 Marines	30	0.4
$2s\_vs\_1sc$	2 Stalkers	1 Spine Crawler	30	0.3
$3s\_vs\_3z$	3 Stalkers	3 Zealots	240	0.6

for example, $2m\_vs\_1z$ contains a rectangular flat brick field, $3s\_vs\_3z$ contains a square of grass. We evaluate our AdvTest on four representative maps as shown in Table I. The second and third columns show the type and number of soldiers controlled by the testing agent and target agent on each map, respectively. We choose these four maps because they contain four types of soldiers and different troop configurations, such as evenly matched (e.g., $3m$, 3 marines on two sides.) and unevenly matched (e.g., $2m\_vs\_1z$, 2 marines on one side and 1 zealot on the other side) troop configurations.

C. Baselines and Evaluation Metrics

1) Baselines: We compare AdvTest with three commonly-used and state-of-the-art techniques.

- **QMIX** [24] is a commonly-used DRL algorithm for training a policy to control multiple soldiers in competitive environments. We use the implementation in PyMARL2 [23] for this baseline.
- **QMIX**$_{noise}$**[25]** is a state-adversarial learning method, which enhances the robustness of QMIX by adding state noise produced by a truncated normal distribution.
- **EMOGI** [12] is the state-of-the-art technique for testing the agent in competitive environments, which aims to generate behavior-diverse game agents by leveraging DRL and evolutionary algorithm. It can generate a population of game agents with diverse strategies to reveal diverse failure scenarios of target agent. Note that the prior work Wuji [11] by the same authors applies similar techniques to find the game bugs (e.g., crash bugs, etc.) automatically. We use this state-of-the-art one as baseline, and conduct experiments with the implementation provided by the authors.
- **BehAVExplor** [10] is the state-of-the-art fuzzing framework for exploring the diverse behaviors of agents in autonomous driving systems. We modify the initial seed set and mutation method to form the variant, which is used as as a baseline to avoid its domain-specific design.

The above baselines are chosen for the following reasons: Due to its superior performance in agent training tasks, QMIX is used as our baseline for discovering agent's failure scenarios. Perturbations in adversarial training is able to change the scenarios the agent faces and therefore serves QMIX$_{noise}$ as the second baseline. EMOGI and BehAVExplor modify the elements in the environment and measure the diversity of agent to discover the diverse failure scenarios of the target agent, and therefore also serve as baselines. In summary, these baselines take into account the number or diversity of generated failure scenarios.

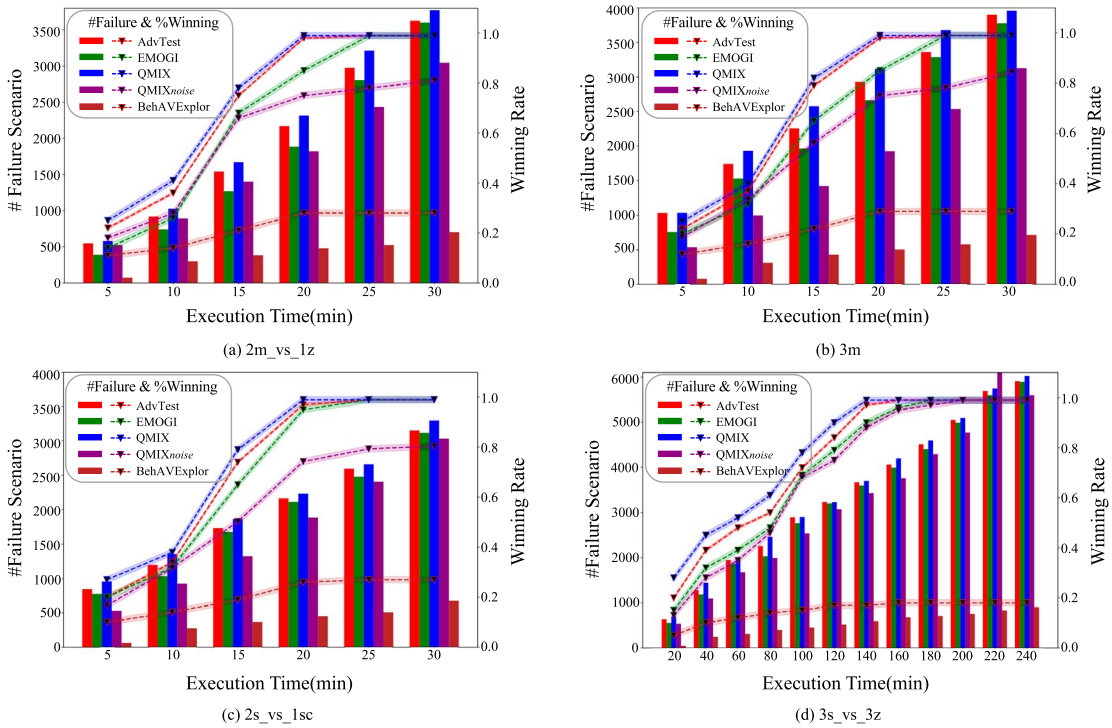
2) Evaluation Metrics: Following previous work [6], [11], [14], [26], [27], we use the following five metrics to evaluate the effectiveness of AdvTest in finding failure scenarios and the diversity of failure scenarios:

- **%Winning**: The winning rate of testing agent.
- **#Failure**: The number of failure scenarios found by each approach.
- **#Distance**: The average distance between all the failure scenarios. Specifically, we calculate the distance between each pair of two failure scenarios based on Eq. 2 and get the average.
- **#Failure**$_u$: The number of unique failure scenarios. We measure the uniqueness by Eq. 2 and the similarity threshold θ_s which will be detailed illustrated in Sec. IV-D.
- **%Coverage**: The ratio of covered states in terms of the whole state space. Following existing studies [27], we divide each dimension of the state space into 5 intervals, and the whole state space is composed of dozens of grids. During the testing, we obtain the states at each time step, and put them in the specific grid. %Coverage is then calculated with the ratio of covered grids to all grids.

D. Experimental Setup

In all experiments, we use AdvTest and baselines to respectively test the target agent within the same testing duration following the common setup [11], [26]. On each map, testing is repeated five times to avoid randomness, and the average performance with minimum and maximum is presented. Due to the different complexity of each map, the testing duration is different. The map containing most soldiers and state sequences with the highest dimensions should have the longest testing duration, e.g., $3s\_vs\_3z$. Hence, the testing duration is set as 30min, 30min, 30min and 240min on four maps as shown in Table I.

We use the basic reinforcement learning algorithm (QMIX) as the basis for training the testing agent to compete against the target agent, and add constraints in the training process. For hyper-parameters, the size of sliding window is set to 200, and the frequency threshold θ_f is set to 0.2. The similarity threshold θ_s should be considered according to the situation of each map since the state sequences generated in each map can have varying sizes. For the size of sliding window and θ_f, we first run AdvTest on each map for a short period, and treat these data as a validation set. Based on it, we determine the parameters. Then we re-run AdvTest for evaluation. For θ_s, the authors first randomly select 100 failure scenarios from each map. Then we check each pair of two sequences, and determine whether they belong to the same failure scenario. For the pairs belonging to the same scenario, we calculate the distance between the two sequences. We use the largest distance as the threshold θ_s. Three authors are involved in the process, and each pair is checked by two of them. When different opinions arise, we engage in discussions until a consensus is reached. The detailed values are shown in Table I.



(a) 2m_vs_1z (b) 3m

(c) 2s_vs_1sc (d) 3s_vs_3z

Fig. 5. Result of winning rate and the number of failure scenarios (RQ1).

We implement AdvTest with PyTorch[2]. All experiments are launched on a server equipped with an NVIDIA TITAN RTX GPU, Intel Xeon Silver CPU, 64GB RAM, running on Ubuntu 20.04 OS.

V. Results and Analysis

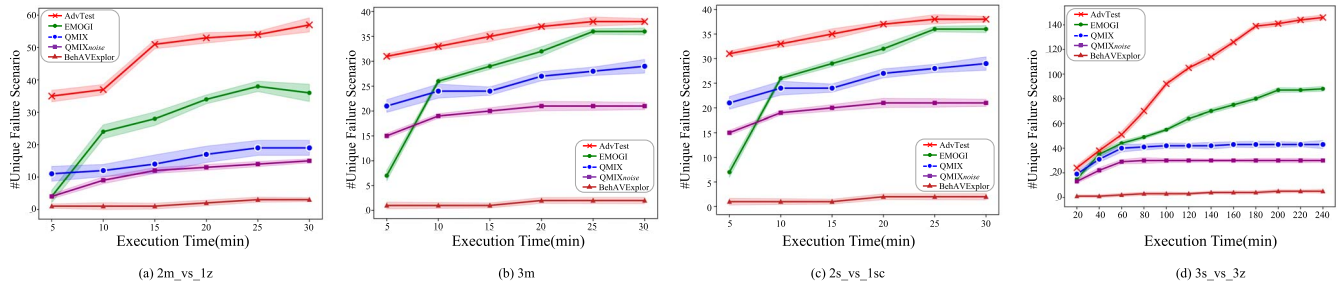
A. RQ1: Effectiveness of AdvTest

1) Number of Failure Scenarios: Fig. 5 presents the trend of winning rate of testing agent (%Winning), and the trend of the total number of failure scenarios of target agent (#Failure) with varying time. Overall, %Winning and #Failure increase over time and finally reach stability. We present and analyze the performance of AdvTest on these two metrics compared with the baselines.

We can see that during the stage where the %Winning goes up (0-25min in Fig. 5(a)–5(c), 0-160min in (d)), QMIX (blue line) shows the highest %Winning, AdvTest (red line) is the second, EMOGI (green line) is the third, QMIX$_{noise}$ (purple line) is the fourth and BehAVExplor (brown line) shows the lowest %Winning. This is because in order to explore the diverse strategies of the target agent, AdvTest adds constraints that prevent the testing agent from taking the most straightforward way to win the battle, thus affecting the %Winning. The goal of the testing agent trained by QMIX is only to win, so the %Winning increases rapidly. In order to explore diversity, EMOGI employs cross-mutation, which results in a huge amount of randomness and therefore severely affects the %Winning. For QMIX$_{noise}$, as described in the existing work [28], although adversarial learning enhances the ability of testing agent to resist attacks and abnormal scenarios which rarely occurs under natural conditions, it also reduces its decision-making ability in the face of normal scenarios due to the unreality of these perturbations, ultimately lowering the winning rate. As for BehAVExplor, under the fuzzing framework, the actions of the testing agent are obtained by applying mutation to a pre-defined sequence of actions, which makes the testing agent have poor decision-making ability. So BehAVExplor has the lowest %Winning. Even though exploring diversity affects the growth of the testing agent's %Winning, the winning rate can reach 100% for QMIX, AdvTest and EMOGI at the late stage of testing. It shows that exploration of diverse strategies will force the testing agent to deliberately take many detours in training, but it can still obtain the winning strategies steadily later.

Throughout the course of testing, QMIX (blue bar) found the most #Failure, AdvTest (red bar) the second, EMOGI (green bar) is the third, QMIX$_{noise}$ (purple bar) is the fourth and BehAVExplor (brown bar) is the least. Similar to %Winning, exploring the strategy diversity of the target agent and enhancing the robustness affect #Failure. In addition, #Failure is also related to the speed of testing execution, when the testing budget (duration) is fixed, the more battles conducted, the more failure scenarios might be generated. Since AdvTest has more overhead due to the need to check diversity and select constraints, fewer battles are conducted in the same time period compared to QMIX, which will be discussed in Sec. VI-C. EMOGI needs to generate a population and cross-mutate the population, which is a huge additional overhead. Therefore, compared with EMOGI,

[2]https://pytorch.org/



(a) 2m_vs_1z (b) 3m (c) 2s_vs_1sc (d) 3s_vs_3z

Fig. 6. Result of the number of unique failure scenarios (RQ1).

TABLE II
RESULT OF AVERAGE DISTANCE SCORE AND STATE COVERAGE (RQ1)

Map Name	2m_vs_1z		3m		2s_vs_1sc		3s_vs_3z	
Method \ Metric	#Distance	%Coverage	#Distance	%Coverage	#Distance	%Coverage	#Distance	%Coverage
QMIX	0.0656	34.67%	0.1720	25.49%	0.0348	46.22%	0.4096	21.07%
QMIX$_{noise}$	0.0694	32.33%	0.1764	25.62%	0.0329	42.76%	0.3816	18.66%
EMOGI	0.1022	50.08%	0.2894	42.17%	0.0711	62.20%	0.5064	39.66%
BehAVExplor	0.0431	4.08%	0.1206	2.69%	0.0279	6.24%	0.3428	1.93%
AdvTest	**0.1924**	**77.21%**	**0.3341**	**61.49%**	**0.1235**	**82.17%**	**0.5929**	**56.41%**

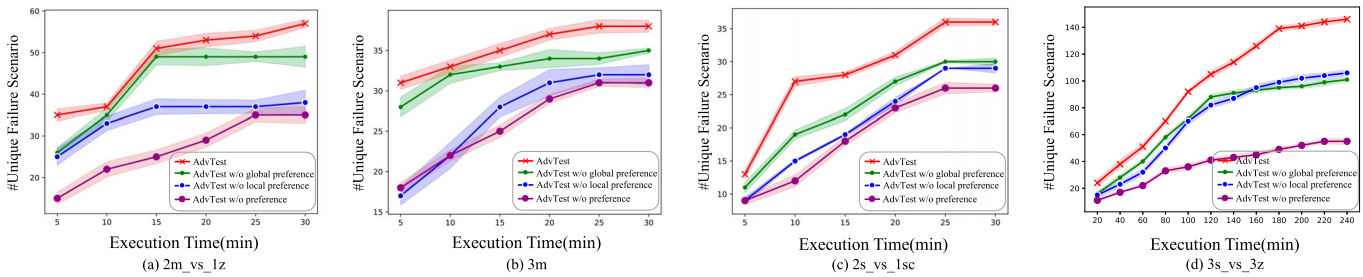
AdvTest has less overhead and can get more #Failure. As mentioned above, the winning rate of QMIX$_{noise}$ and BehAVExplor for testing agent is low, resulting in fewer failure scenarios.

2) Diversity of Failure Scenarios: ❶ **The number of unique failure scenarios:** Fig. 6 shows the trend of the number of unique failure scenarios (i.e., #Failure$_u$) found by these four approaches. Overall, AdvTest (red line) outperforms baselines at any time after the start of testing. From Fig. 6(a), 6(b), and 6(d), we can see that for a period of time after the testing began, EMOGI (green line) performs worse than QMIX (blue line). As mentioned in Sec. V-A1, EMOGI produces fewer failure scenarios at the early stage, therefore fewer #Failure$_u$. Later, under the combined effect of cross-mutation and DRL, EMOGI found more unique failure scenarios than QMIX. As for QMIX$_{noise}$, the perturbation increases the type of scenario encountered by the testing agent, but it is often some abnormal state that is not real. When testing agent interacts with the target agent, its robustness in the face of anomalies hardly helps it to explore the various failure scenarios of the target agent in real scenarios, so the number of unique failure scenarios does not increase. Furthermore, BehAVExplor generates the fewest failure scenarios throughout the testing (mentioned in Sec. V-A1). As a result, it generated the fewest unique failure scenarios.

We can see that #Failure$_u$ rises rapidly in the early stage and reachs a plateau in the later stage, meaning that almost no new unique failure scenarios are found. It is worth noting that AdvTest tends to be the last one to reach the plateau compared to the baselines, which shows that AdvTest can discover unique failure scenarios continuously even when other methods cannot boost the diversity anymore. When the testing finishes, AdvTest totally found 57, 38, 36 and 146 unique failure scenarios on the four maps, while the other four baselines are 36, 36, 29, 88 (EMOGI), 19, 29, 21, 43 (QMIX), 15, 21, 15, 30 (QMIX$_{noise}$) and 3, 2, 2, 5 (BehAVExplor). Compared with the best baseline (EMOGI), AdvTest has increased by 58.3%, 5.6%, 24.1% and 65.9% on the four maps respectively. We note that although AdvTest finds not the most #Failure as shown in Fig. 5, AdvTest finds the most #Failure$_u$ due to adding constraints.

❷ **Average distance score and state coverage of failure scenarios:** Table II shows the average distance score and state coverage of failure scenarios (i.e., #Distance and %Coverage) generated by AdvTest and baselines. A larger value indicates a larger distance between failure scenarios or larger state coverage, that is, more diverse failure scenarios. Compared with the best baseline (EMOGI), #Distance obtained by AdvTest increases by 88.3%, 15.4%, 73.7%, and 17.1%, and %Coverage obtained by AdvTest increases by 54.2%, 45.8%, 32.1%, and 42.2% on the four maps, respectively.

Furthermore, we analyze the statistically significant of results. Specifically, we propose hypotheses and calculate the p-value on the four baselines and four maps respectively. The hypothesis is: There are larger distance between the failure scenarios generated by AdvTest (i.e., AdvTest has higher diversity than baselines). We name the set of failure scenarios generated by AdvTest and a baseline FS_1 and FS_2. Firstly, we randomly select two failure scenarios from FS_1 and FS_2 respectively. Then, the distance between each pair of failure scenarios is defined as dis_1 and dis_2. The process is repeated for 500 times. Finally, we get the p-value based on the probability of event $(dis_2 > dis_1)$ to verify the truth of the hypothesis. As Table III shows, the p-values are all low, with the maximum of 0.017. The lower p-value, the greater the probability of conforming



(a) 2m_vs_1z (b) 3m (c) 2s_vs_1sc (d) 3s_vs_3z

Fig. 7. Result of ablation study (RQ2).

TABLE III
RESULT OF STATISTICAL HYPOTHESIS TESTS (RQ1)

Map Name	2m_vs_1z	3m	2s_vs_1sc	3s_vs_3z
QMIX-AdvTest	<0.01	<0.01	<0.01	0.012
QMIX$_{noise}$-AdvTest	<0.01	0.01	<0.01	<0.01
EMOGI-AdvTest	0.012	0.015	0.011	0.017
BehAVExplor-AdvTest	<0.01	<0.01	<0.01	<0.01

to the proposed hypothesis, that is, the larger the distance between failure scenarios in FS_1. As a result, the performance of AdvTest has significant advantages.

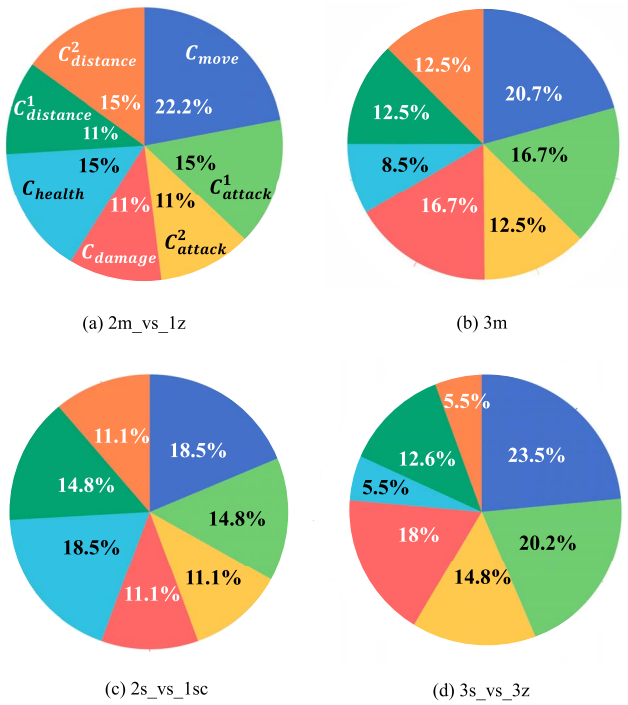
Answer to RQ1: Compared with the commonly-used and state-of-the-art techniques, AdvTest can find more and diverse failure scenarios during the same testing period.

B. RQ2: Ablation Study

Fig. 7 shows the trend of #Failure$_u$ found by AdvTest and three variants (removing local preference, global preference, and both) during testing. AdvTest outperforms the other three variants throughout the testing process. This suggests that both parts of AdvTest (*global preference* & *local preference*) contribute to the constraint selection.

In the early stage of testing, *AdvTest w/o global preference* (green line) performs much better than *AdvTest w/o local preference* (blue line). The gap between them gets smaller and smaller as the testing goes on. In the early stage of testing, the global preference (mentioned in Sec. III-D1) of each constraint is 0, indicating the constraints are randomly selected, as none of the constraints have been selected. The poor performance of *AdvTest w/o local preference* in the early stage is due to such cold start issue. Specifically, since the probability of selecting each constraint is determined by the average value of the diversity gain brought by each constraint adding, this average value is more reflective of the ability to explore diversity when constraints are added more times. Thus, with time going on, the constraints selected only based on the global preference become more reasonable.

AdvTest w/o global preference (green line) outperforms *AdvTest w/o preference* (purple line) throughout testing. As mentioned above, due to the cold start, *AdvTest w/o local preference* (blue line) performed similarly to *AdvTest w/o preference* at the beginning and then outperformed *AdvTest w/o preference* as more and more constraint selections are made.



(a) 2m_vs_1z (b) 3m (c) 2s_vs_1sc (d) 3s_vs_3z

Fig. 8. The proportion of selected times per constraint (RQ3).

Answer to RQ2: Both global and local preference are useful for constraint selection. Local preference is useful all the time, while the global preference is more useful in the late stages.

C. RQ3: Necessity of Each Constraint

Fig. 8 demonstrates the proportion of the number of times each constraint is selected (%Selection) on the four maps. Generally speaking, all the constraints are selected on all the experimental maps, indicating the necessity of all derived constraints. Besides, as we mentioned before, some constraints might have similar effects, e.g., "attack within smaller range" (C^1_{attack}) and "smaller distance between two-sided agents" ($C^1_{distance}$)

P1: Z initiates a simultaneous assault, launching attacks on both M_L and M_R . In the ensuing battle, Z successfully kills M_L but ultimately falls victim to M_R's retaliation.

P2: Z launches an attack on M_R while M_L swiftly retreats and distances itself from Z. Once Z kills M_R , both M_L and Z come to a standstill, ceasing any further attacks. After a brief pause, M_L initiates a movement and successfully kills Z.

Fig. 9. Two patterns in Category 1 (one marine survives, small health difference).

would both encourage the agents battle in a close manner. Nevertheless, the results show that both constraints are selected with a relatively large number of times, which again implies the necessity of each constraint.

Answer to RQ3: All the constraints are selected on all the experimental maps, indicating the necessity of each derived constraints.

VI. Discussion

A. What Battle Patterns Can AdvTest Find?

We manually analyze the unique failure scenarios generated on $2m\_vs\_1z$ by AdvTest and four baselines, to provide a high-level viewpoint about the battle patterns. Specifically, on $2m\_vs\_1z$ map, the testing agent controls two marines (M_L and M_R) and the target agent controls a zealot (Z). A Marine can attack the opponent within a given attacking range, while a Zealot can only attack the opponent in close proximity. The grid above the soldier's head indicates the remaining health value, where the green indicates a large health value, and the orange indicates a small health value. We summarize four categories and total of eight battle patterns (Figs. 9–12), as follows.

- Category 1: ***One*** marine survives, and it wins by a narrow margin (i.e., ***small*** health difference). This category involves two patterns as demonstrated in Fig. 9.
- Category 2: ***One*** marine survives, and it wins a landslide victory (i.e., ***large*** health difference). This category involves two patterns as demonstrated in Fig. 10.
- Category 3: ***Two*** marines survive, and they win by a narrow margin (i.e., ***small*** health difference). This category involves one pattern as demonstrated in Fig. 11.
- Category 4: ***Two*** marines survive, and they win a landslide victory (i.e., ***large*** health difference). This category involves three patterns as demonstrated in Fig. 12.

Table IV demonstrates the statistics of eight battle patterns found by AdvTest and the baselines. We can see that, for patterns P2, P4, P5, P6, P8, only our proposed AdvTest can generate the failure scenarios involving such patterns. The numbers in Table IV indicate the number of unique failure scenarios in each specific battle patterns, where the failure scenarios per pattern might demonstrate difference in actual attack range, damage value of some attack, health difference during some period, etc. We further discuss the reasons why AdvTest can generate these new patterns that are not covered by the baselines. Note that because there are several constraints being added during testing, the generated failure patterns may be related to multiple constraints.

P2: $C^2_{distance}$ causes M_L to be far away from Z. Furthermore, due to the influence of C_{health}, the testing agent adopts a conservative attack strategy, leading to a small difference in health value between M_L and Z. However, if the testing agent's objective is to win, i.e., without any constraints, M_L would initiate an attack on Z at the beginning instead of retreating.

P4: C_{move} restricts the movement range of M_R, while $C^2_{distance}$ ensures that M_R maintains a safe distance from Z. As a result of these constraints, M_R does not retaliate against Z initially. So Z focuses on killing M_L and avoids being attacked by both sides.

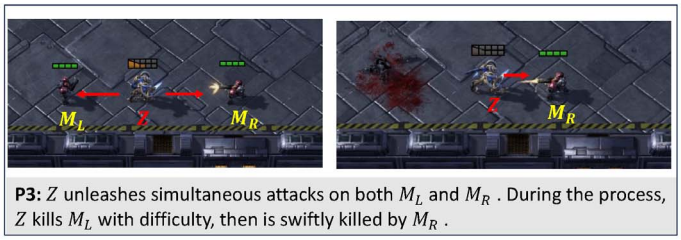
P5: The reasons for smaller movement range and smaller health difference are similar with **P4**. In addition, due to the constraint $C^2_{distance}$, M_L retreats or runs away from the opponent, which causes Z to shift its focus and start attacking M_R instead.

P6: Under the influence of $C^2_{distance}$ and C_{damage}, the distance between M_L and Z is very large and M_L moves back and forth without engaging in any attacks.

P8: C^2_{attack} can enlarge the attack range of M_L and M_R, which can make them attack from a distance. Under the effect of C^2_{attack}, the distance between two-sided agents tends to be large, so M_L and M_R choose to run away when Z tends to approach them. They choose to attack from a distance to protect themselves from harm.

B. Analysis of Different Parameters

As Fig. 13 shows, we take $2m\_vs\_1z$ as an example and discuss the impact of different hyper-parameters (the similarity threshold θ_s, the frequency threshold θ_f and the size of sliding window) on the performance.



P3: Z unleashes simultaneous attacks on both M_L and M_R . During the process, Z kills M_L with difficulty, then is swiftly killed by M_R .

P4: Z hovers for a while then kills M_L with difficulty. Due to the limited range of M_R 's attacks, M_R is unable to retaliate against Z initially. However, as Z makes a move, M_R seizes the opportunity and successfully kills Z .

Fig. 10. Two patterns in Category 2 (one marine survives, large health difference).

P5: Z hovers around before targeting M_L. However, M_L evades, causing Z to shift focus to M_R. Simultaneously, M_L retaliates, forcing Z to defend against M_L 's counterattacks. Z is killed while in motion, leaving both M_L and M_R on the brink of death.

Fig. 11. One pattern in Category 3 (two marines survive, small health difference).

P6: Z initiates an attack on M_R , while M_L moves back and forth without engaging in any attacks. As M_R 's health diminishes, M_L seizes the opportunity to retaliate against Z . In an attempt to defend M_L , Z opts to move left but is killed while in motion.

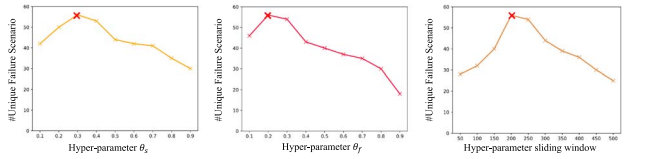
P7: M_L and M_R take turns launching attacks on Z, resulting in a back-and-forth dynamic. Throughout this process, Z remains at a distance from M_L and M_R, refraining from causing them large harm.

P8: M_L and M_R engage in a pattern of alternating attacks on Z from a safe distance, quickly retreating afterwards. As a result, Z is required to cover more ground in order to reach and retaliate against M_L and M_R.

Fig. 12. Three patterns in Category 4 (two marines survive, large health difference).

TABLE IV
STATISTICS ABOUT BATTLE PATTERNS FOUND BY ADVTEST AND BASELINES

Method	Categroy1		Categroy2		Categroy3	Categroy4			Sum
	P1	P2	P3	P4	P5	P6	P7	P8	
QMIX	2	×	4	×	×	×	13	×	19
QMIX$_{noise}$	1	×	2	×	×	×	12	×	15
EMOGI	12	×	10	×	×	×	14	×	36
BehAVExplor	3	×	×	×	×	×	×	×	3
AdvTest	15	3	11	4	2	3	16	3	57



(a) Different settings of θ_s (b) Different settings of θ_f (c) Different settings of sliding window

Fig. 13. Result of different hyper-parameter settings.

We can see that as the parameters increase, #Failure$_u$ first increases and then decreases. For θ_s, the reason is that when the value is low, more failure scenarios are judged as never occurring, and AdvTest would have the illusion of "good diversity", and thus will not add constraints. When the value is high, many failure scenarios are considered to have occurred, and thus AdvTest will frequently add constraints. This would cause the testing agent to be frequently disturbed by constraints, and cannot adequately train its policy network. Then it is difficult for testing agent to defeat the target agent and thus unable to obtain unique failure scenarios.

Similarly, the lower θ_f results in constraints not being added when they should be. Higher θ_f results in inadequate training of the testing agent. For the sliding window, a value that is too large or too small does not reflect the agent's recent behavior. Specifically, larger window contains a great many failure scenarios, and smaller window can not contain enough information.

C. Threats to Validity

The first threat concerns about the generality of the proposed approach. Although we only conduct experiments on four maps of StarCraft II, these four maps are quite different, involving different types of soldiers and terrains, and we repeat the experiment multiple times. This could alleviate the threat to some extent. For constraint construction, the constraints are based on the general design concepts of the competitive games, and reflect the common situations of in the game regression. These constraints can usually be easily extracted by learning the agent descriptions and the game knowledge in the game manual and user community. If no manual is available, users can also grasp the information of the agent by watching the game video or trying to experience a few games, so as to know about the action space and the attributes of the agent. For example, for users who do not know Starcraft II, they can quickly learn about the agent's health, attack ability and other information in the game manual or just by playing a few games. Three authors together construct the constraints and take about half day. For the adaption, many constraints can be reused due to the game similarity. Let's take another competitive game, Google Football [29], as an example to further explain the process of constraint set construction. In this environment, agent must control the players, learn how to pass the ball between them, and how to overcome the opponents' defenses to score goals. Based on the general concepts and game introduction, we can design constraints based on the speed of movement, the shooting range, the movement range of footballer, the type of passing (long pass, short pass, high pass) which is obtained from the action set of footballer. In this way, the constraint set of Google Football can be as follows:

1) **Smaller movement speed** C^1_{speed}: constrain the movement speed of agent to less than 50% of its default speed.
2) **Larger movement speed** C^2_{speed}: constrain the movement speed of agent to 150% of its default speed.
3) **Near shooting range of footballers** $C^1_{shooting}$: constrain footballers to shooting only in the penalty area.
4) **Far shooting range of footballers** $C^2_{shooting}$: constrain footballers to shooting only outside the penalty area.
5) **Movement within smaller range** C_{move}: constrain the bound of the movement range to 50% of its default value.
6) **No long pass** C^1_{pass}: constrain footballers not long pass.
7) **No short pass** C^2_{pass}: constrain footballers not short pass.
8) **No high pass** C^3_{pass}: constrain footballers not high pass.

Secondly, we utilize a normalized version of Hamming distance to measure the similarity between two state sequences of different lengths in *Diversity Check*, which mainly considers the forehead common segments of two sequences. Different distance metrics might influence the measurement of unique failure scenarios, and the promising results and the former practice [10] tackling similar issue might reduce the threat to some extent. Additionally, we use multiple metrics (i.e., #Failure$_u$, #Distance, %Coverage), and conduct manual evaluation to evaluate the performance (diversity) of AdvTest. The results of manual evaluation and automated evaluation show consistency, which mutually demonstrated the correctness of evaluation and the effectiveness of AdvTest.

The third threat is computational overhead from *Diversity Check* and *Constraint Selection*. We believe the computational cost caused by the modules in AdvTest is acceptable. Specifically, we additionally counted the number of battles (#Battle) (i.e., test cases) performed by each method besides #Failure and #Failure$_u$ during the same testing period in our experiment. As shown in Table V, we analyze the computational cost in AdvTest by comparing it with the DRL algorithm QMIX which has the best computational efficiency among the baselines. For example, AdvTest performed 3.5% fewer battles and produced 3.9% fewer failure scenarios, but the number of unique failure scenarios increased by 200% on map $2m\_vs\_1z$. Similar results are shown on other maps.

The fourth threat is that battle patterns in Section VI-A are identified and analyzed with human efforts. StarCraft II is a classic game for both amusement and academic research with rich handbooks and player communities. We have conducted careful study on these materials, which help deepen the understanding of the game.

TABLE V
ANALYSIS OF COMPUTATIONAL COST IN ADVTEST

	#Battle	#Failure	#Failure$_u$
2m_vs_1z	↓3.5%	↓3.9%	↑200%
3m	↓2.2%	↓2.5%	↑31.0%
2s_vs_1sc	↓3.6%	↓4.1%	↑71.4%
3s_vs_3z	↓1.7%	↓2.0%	↑239.5%

VII. RELATED WORK

A. Testing for MDP-Based AI Model

Autonomous Driving System (ADS) is an typical scenario based on MDP. There are two main types of methods for testing ADSs: knowledge-based [30], [31], [32], [33], [34] and search-based [10], [35]. Knowledge-based methods use domain-specific ontologies or metamorphosis relationships to generate critical cases that are designed based on human driving experience, traffic accident reports, and traffic laws and regulations [36]. Zhou et al. [30] generated critical cases by adding adversarial perturbations to real billboards to check whether the model maintained the same output of steering angle. Zhang et al. [31] and Tian et al. [32] generated extreme weather and tested whether ADS worked properly under such conditions. Search-based methods typically find a set of test parameters that lead to behavioral differences in ADS, such as collisions and lane departures. For example, Cheng et al. [10] discovered diverse violations by mutation. Gambi et al. [35] used genetic algorithms to evaluate parameters such as the length and position of the road, making the ego-vehicle more error-prone in the lane-keeping task in the generated test cases. Pang et al. [14] proposed the first black-box fuzz testing framework MDPFuzz to test intelligent software in more MDP scenarios. Although this work can cover more continuous decision-making scenarios, only the initial state of the sequence is mutated during the generation of test cases, which limits the performance of the method. In this work, the adversarial strategy is adopted to avoid unreasonable mutation and to realize the perturbation of the intermediate state of the environment. Additionally, these methods do not work well in our scenario due to different application contexts.

B. Adversarial Attack on Deep Reinforcement Learning

Many studies have found that Deep Neural Networks (DNNs) are vulnerable to adversarial attacks [37], [38], [39], [40], where an attacker can interfere with the data inputs (e.g., images) of a DNN by generating adversarial samples, thus forcing the network to incorrectly categorize the interfered data. Recently, such adversarial attacks have been extended and targeted at DRL, or more precisely, at policy networks of trained agents. The attack methods against DRL can be divided into three categories: attack by observation manipulation, attack by action/trajectory manipulation, and attack by adversarial agents. Huang et al. [4] and Behzadan & Munir [5] first proposed to perturb the victim's observations at each time step to disrupt the victim's decision making and force its policy network to output sub-optimal behaviors, thus completing the attack. In recent researches [41], [42], an attacker can determine the best time step to launch an attack, minimizing efforts to manipulate the environment. Pan et al. [43] and Lee et al. [44] demonstrated that action manipulation can interfere with the DRL's environment, causing the agent to fail in a specific way, which is more useful in practical applications. Gleave et al. [6] proposed to introduce an adversarial agent in the environment. The adversary can create natural observations that can act as adversarial inputs and make the adversary attack the victim successfully. Guo et al. [8] and Wu et al. [7] reconstructed the loss function of the DRL algorithm to make the adversarial agent more capable of exploiting the weaknesses of the victim agent. In terms of the problem setup, the work most relevant to ours is the attacks through adversarial agents. Since their goal is to win, agents tend to battle in a way that maximizes their winning rate, neglecting to explore the diversity of strategies. In our approach, the testing agent explores the strategy diversity of the target agent while revealing its weaknesses.

VIII. CONCLUSION

In complex competitive scenarios, the agent must not only adapt to a static set of rules, but also to the changing strategies of the opponent, so there is an urgent need for testing the agent's capabilities. In this paper, we propose a diversity-oriented framework AdvTest for testing agent of competitive games with the constraint-guided adversarial agent training. First, we summarize seven constraints through the game manual and user discussions to address *what are the suitable constraints*. Then, we design two modules, *Diversity Check* and *Constraint Selection*, respectively, to address *when should we introduce constraints during adversarial agent training* and *which constraint should be added at specific time*. The experimental results show that our method is able to find more and diverse failure scenarios compared to baselines. We also summarized eight types of battle patterns and the results show that AdvTest can reveal more battle patterns. Future work will utilize the idea of "constraint-guided" to boost the testing diversity of other AI systems.

ACKNOWLEDGEMENT

Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore and Cyber Security Agency of Singapore.

REFERENCES

[1] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, pp. 982–987, 2023.
[2] S. E. Li, "Deep reinforcement learning," in *Reinforcement Learning for Sequential Decision and Optimal Control,* Singapore: Springer, 2023, pp. 365–402.
[3] M. Lanctot et al., "A unified game-theoretic approach to multiagent reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst. 30: Annu. Conf. Neural Inf. Process. Syst.*, Long Beach, CA, USA, I. Guyon, U. von Luxburg, S. Bengio, H. M. Wallach, R. Fergus, S. V. N. Vishwanathan, and R. Garnett, Eds., 2017, pp. 4190–4203.
[4] S. H. Huang, N. Papernot, I. J. Goodfellow, Y. Duan, and P. Abbeel, "Adversarial attacks on neural network policies," *CoRR*, vol. abs/1702.02284, 2017. [Online]. Available: http://arxiv.org/abs/1702.02284

[5] V. Behzadan and A. Munir, "Vulnerability of deep reinforcement learning to policy induction attacks," in *Proc. Mach. Learn. Data Mining Pattern Recognit. - 13th Int. Conf. (MLDM)*, New York, NY, USA, P. Perner, Ed., vol. 10358, New York, NY, USA: Springer, 2017, pp. 262–275.

[6] A. Gleave, M. Dennis, C. Wild, N. Kant, S. Levine, and S. Russell, "Adversarial policies: Attacking deep reinforcement learning," in *Proc. 8th Int. Conf. Learn. Representations (ICLR),* Addis Ababa, Ethiopia, 2020.

[7] X. Wu, W. Guo, H. Wei, and X. Xing, "Adversarial policy training against deep reinforcement learning," in *Proc. 30th USENIX Security Symposium, USENIX Security 2021*, Virtual Event, M. Bailey and R. Greenstadt, Eds. USENIX Association, 2021, pp. 1883–1900.

[8] W. Guo, X. Wu, S. Huang, and X. Xing, "Adversarial policy learning in two-player competitive games," in *Proc. 38th Int. Conf. Mach. Learn., (ICML)*, Virtual Event, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 2021, pp. 3910–3919.

[9] T. V. Bui, T. Mai, and T. H. Nguyen, "Imitating opponent to win: Adversarial policy imitation learning in two-player competitive games," in *Proc. Int. Conf. Auton. Agents Multiagent Syst., (AAMAS)*, London, United Kingdom, N. Agmon, B. An, A. Ricci, and W. Yeoh, Eds. New York, NY, USA: ACM, 2023, pp. 1285–1293.

[10] M. Cheng, Y. Zhou, and X. Xie, "Behavexplor: Behavior diversity guided testing for autonomous driving systems," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Testing Anal., (ISSTA)*, Seattle, WA, USA, R. Just and G. Fraser, Eds., New York, NY, USA: ACM, 2023, pp. 488–500.

[11] Y. Zheng et al., "Wuji: Automatic online combat game testing using evolutionary deep reinforcement learning," in *Proc. 34th IEEE/ACM Int. Conf. Automated Softw. Eng., (ASE)*, San Diego, CA, USA, Piscataway, NJ, USA: IEEE Press, 2019, pp. 772–784.

[12] R. Shen et al., "Generating behavior-diverse game AIS with evolutionary multi-objective deep reinforcement learning," in *Proc. 29th Int. Joint Conf. Artif. Intell. (IJCAI)*, C. Bessiere, Ed. ijcai.org, 2020, pp. 3371–3377.

[13] Z. Tang et al., "Discovering diverse multi-agent strategic behavior via reward randomization," *CoRR*, vol. abs/2103.04564, 2021. [Online]. Available: https://arxiv.org/abs/2103.04564

[14] Q. Pang, Y. Yuan, and S. Wang, "Mdpfuzz: testing models solving markov decision processes," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*, Virtual Event, South Korea, S. Ryu and Y. Smaragdakis, Eds., New York, NY, USA: ACM, 2022, pp. 378–390.

[15] S. Hu, C. Xie, X. Liang, and X. Chang, "Policy diagnosis via measuring role diversity in cooperative multi-agent RL," in *Proc. Int. Conf. Mach. Learn.*, ICML 2022, 17–23 July 2022, Baltimore, Maryland, USA, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvári, G. Niu, and S. Sabato, Eds., vol. 162, PMLR, 2022, pp. 9041–9071.

[16] O. Vinyals et al., "Grandmaster level in starcraft II using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.

[17] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *Proc. 34th Int. Conf. Mach. Learn., (ICML)*, Sydney, NSW, Australia, D. Precup and Y. W. Teh, Eds., vol. 70, PMLR, 2017, pp. 22–31.

[18] K. Srinivasan, B. Eysenbach, S. Ha, J. Tan, and C. Finn, "Learning to be safe: Deep RL with a safety critic," 2020, *arXiv:2010.14603*.

[19] D. J. Berndt and J. Clifford, "Using dynamic time warping to find patterns in time series," in *Proc. Knowl. Discovery Databases: Papers from AAAI Workshop*, U. M. Fayyad and R. Uthurusamy, Eds., Seattle, WA, USA: AAAI Press, 1994, pp. 359–370.

[20] R. W. Hamming, *Coding and Information Theory, 2nd ed.*. Prentice Hall, 1986.

[21] V. Kuleshov and D. Precup, "Algorithms for multi-armed bandit problems," 2014, *arXiv:1402.6028*.

[22] M. Tokic and G. Palm, "Value-difference based exploration: Adaptive control between epsilon-greedy and softmax," in *Proc. Adv. Artif. Intell., 34th Annu. German Conf. AI*, Berlin, Germany, J. Bach and S. Edelkamp, Eds., vol. 7006, Berlin, Germany: Springer, 2011, pp. 335–346.

[23] J. Hu, S. Jiang, S. A. Harding, H. Wu, and S.-w. Liao, "Rethinking the implementation tricks and monotonicity constraint in cooperative multi-agent reinforcement learning," 2021, *arXiv:2102.03479*.

[24] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. N. Foerster, and S. Whiteson, "QMIX: monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proc. 35th Int. Conf. Mach. Learn., (ICML)*, Stockholmsmässan, Stockholm, Sweden, J. G. Dy and A. Krause, Eds., vol. 80. PMLR, 2018, pp. 4292–4301.

[25] S. Han et al., "What is the solution for state-adversarial multi-agent reinforcement learning?" 2022, *arXiv:2212.02705*.

[26] F. U. Haq, D. Shin, and L. C. Briand, "Many-objective reinforcement learning for online testing of DNN-enabled systems," in *Proc. 45th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, Melbourne, Australia. Piscataway, NJ, USA: IEEE Press, 2023, pp. 1814–1826.

[27] Z. Li et al., "Generative model-based testing on decision-making policies," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Luxembourg. Piscataway, NJ, USA: IEEE Press, 2023, pp. 243–254.

[28] W. Guo, G. Liu, Z. Zhou, L. Wang, and J. Wang, "Enhancing the robustness of QMIX against state-adversarial attacks," *Neurocomputing*, vol. 572, 2024, Art. no. 127191, doi: 10.1016/j.neucom.2023.127191.

[29] K. Kurach et al., "Google research football: A novel reinforcement learning environment," in *Proc. 34th AAAI Conf. Artif. Intell., AAAI, 32nd Innovative Appl. Artif. Intell. Conf. (IAAI), 10th AAAI Symp. Educ. Adv. Artif. Intell. (EAAI)*, New York, NY, USA: AAAI Press, 2020, pp. 4501–4510, doi: 10.1609/aaai.v34i04.5878.

[30] H. Zhou et al., "Deepbillboard: systematic physical-world testing of autonomous driving systems," in *Proc. 42nd Int. Conf. Softw. Eng.,* Seoul, South Korea, G. Rothermel and D. Bae, Eds., New York, NY, USA: ACM, 2020, pp. 347–358.

[31] M. Zhang, Y. Zhang, L. Zhang, C. Liu, and S. Khurshid, "Deeproad: Gan-based metamorphic testing and input validation framework for autonomous driving systems," in *Proc. 33rd ACM/IEEE Int. Conf. Automated Softw. Eng. (ASE)*, Montpellier, France, M. Huchard, C. Kästner, and G. Fraser, Eds. New York, NY, USA: ACM, 2018, pp. 132–142.

[32] Y. Tian, K. Pei, S. Jana, and B. Ray, "Deeptest: automated testing of deep-neural-network-driven autonomous cars," in *Proc. 40th Int. Conf. Softw. Eng. (ICSE)*, Gothenburg, Sweden, M. Chaudron, I. Crnkovic, M. Chechik, and M. Harman, Eds., New York, NY, USA: ACM, 2018, pp. 303–314.

[33] J. Chandrasekaran, Y. Lei, R. Kacker, and D. R. Kuhn, "A combinatorial approach to testing deep neural network-based autonomous driving systems," in *Proc. 14th IEEE Int. Conf. Softw. Testing, Verification Validation Workshops (ICST)*, Porto de Galinhas, Brazil. Piscataway, NJ, USA: IEEE Press, 2021, pp. 57–66.

[34] A. Gambi, T. Huynh, and G. Fraser, "Automatically reconstructing car crashes from police reports for testing self-driving cars," in *Proc. 41st Int. Conf. Softw. Eng.: Companion Proc., (ICSE)*, Montreal, QC, Canada, J. M. Atlee, T. Bultan, and J. Whittle, Eds., Piscataway, NJ, USA: IEEE Press, 2019, pp. 290–291.

[35] A. Gambi, M. Müller, and G. Fraser, "Asfault: testing self-driving car software using search-based procedural content generation," in *Proc. 41st Int. Conf. Softw. Eng.: Companion Proc., (ICSE)*, Montreal, QC, Canada, J. M. Atlee, T. Bultan, and J. Whittle, Eds. Piscataway, NJ, USA: IEEE Press, 2019, pp. 27–30.

[36] G. Lou, Y. Deng, X. Zheng, M. Zhang, and T. Zhang, "Testing of autonomous driving systems: where are we and where should we go?" in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, Singapore, Singapore, A. Roychoudhury, C. Cadar, and M. Kim, Eds. New York, NY, USA: ACM, 2022, pp. 31–43.

[37] N. Carlini and D. A. Wagner, "Towards evaluating the robustness of neural networks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, San Jose, CA, USA. Los Alamitos, CA, USA: IEEE Comput. Soc. Press, 2017, pp. 39–57.

[38] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*.

[39] N. Papernot, P. D. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, and A. Swami, "The limitations of deep learning in adversarial settings," in *Proc. IEEE Eur. Symp. Secur. Privacy, EuroS&P*, Saarbrücken, Germany. Piscataway, NJ, USA: IEEE Press, 2016, pp. 372–387.

[40] C. Szegedy et al., "Intriguing properties of neural networks," 2013, *arXiv:1312.6199*.

[41] J. Kos and D. Song, "Delving into adversarial attacks on deep policies," in *Proc. 5th Int. Conf. Learn. Representations (ICLR)*, Toulon, France, 2017.

[42] Y. Lin, Z. Hong, Y. Liao, M. Shih, M. Liu, and M. Sun, "Tactics of adversarial attack on deep reinforcement learning agents," in *Proc. 26th Int. Joint Conf. Artif. Intell., (IJCAI)*, Melbourne, Australia, C. Sierra, Ed., 2017, pp. 3756–3762.

[43] X. Pan et al., "Characterizing attacks on deep reinforcement learning," in *Proc. 21st Int. Conf. Auton. Agents Multiagent Syst., (AAMAS)*, Auckland, New Zealand, P. Faliszewski, V. Mascardi, C. Pelachaud, and M. E. Taylor, Eds., 2022, pp. 1010–1018.

[44] X. Y. Lee, S. Ghadai, K. L. Tan, C. Hegde, and S. Sarkar, "Spatiotemporally constrained action space attacks on deep reinforcement learning agents," in *Proc. 34th AAAI Conf. Artif. Intell. (AAAI), 32nd Innovative Appl. Artif. Intell. Conf. (IAAI), 10th AAAI Symp. Educ. Adv. Artif. Intell., (EAAI)*, New York, NY, USA: AAAI Press, 2020, pp. 4577–4584.